📝 Notes App (Backend APIs)

Project Overview

Build a small backend application for a multi-user notes service. Think backend server for Google Keep or Apple Notes. It should expose REST APIs to manage users and their personal notes. The primary features should include:

- New User Registration
- User authentication (Login)
- Creating, reading, updating, and deleting notes
- Sharing a note with another user

Expected Deliverables

- **A working backend server with REST APIs for the above functionality hosted online.** You can use free platforms like Heroku, Railway.app, Render.com, or Fly.io.

Feature Requirements

1. Register New User

Endpoint: `POST /register`

Payload:

JSON

```
None
 {
  "email": "string",
  "password": "string"
}
```

Response: Status code 201 CREATED with a success message.

2. User Authentication (Login)

Endpoint: `POST /login`

Payload:

JSON

```
None
{
  "email": "string",
  "password": "string"
}
```

Response: On success (Status Code: 200 OK), a JWT token is returned.

JSON

```
None
{
  "access_token": "string"
}
```

On failure (Status Code: 401 Unauthorized):

JSON

```
None
{
  "message": "Invalid email or password"
}
```

---

3. Get All Notes for Authenticated User

Endpoint: `GET /notes`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Response: Status code 200 OK with a list of all notes created by the user.

JSON

```
None
[{
  "id": "string",
  "title": "string",
  "content": "string",
  "created_at": "datetime",
  "updated_at": "datetime"
}...]
```

4. Get a Specific Note by ID

Endpoint: `GET /notes/{id}`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Response: Status code 200 OK with the note data. The user should only be able to access their own notes.

---

5. Create a New Note

Endpoint: `POST /notes`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Payload:

JSON

```
None
{
```

```
  "title": "string",
  "content": "string"
}
```

Response: Status code 201 CREATED with the newly created note data.

JSON

```
None
{
  "id": "string",
  "title": "string",
  "content": "string",
  "created_at": "datetime",
  "updated_at": "datetime"
}
```

---

6. Update an Existing Note

Endpoint: `PUT /notes/{id}`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Payload:

JSON

```
None
{
  "title": "string",
  "content": "string"
}
```

Response: Status code 200 OK with the updated note data.

---

7. Delete a Note

Endpoint: `DELETE /notes/{id}`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Response: Status code 204 No Content.

---

8. Share a Note with Another User

Endpoint: `POST /notes/{id}/share`

Header:

```
None
"Authorization": "Bearer <your_jwt_token>"
```

Payload:

JSON

```
None
{
  "share_with_email": "string"
}
```

Response: Status code 200 OK with a success message. After sharing, the user specified in `share_with_email` should be able to access this note via the `GET /notes/{id}` endpoint.

---

9. API documentation

Endpoint: `GET /openapi.json`

Response: a json file containing all endpoints exposed. Use the following structure for the json https://swagger.io/docs/specification/v3_0/basic-structure/.

---

10. About

Endpoint: `GET /about`

Response:

```
None
{
  "name": "your name",
  "email": "your email",
  "my features": {
    "feature name": "Feature description. Why did you choose it."
  }
}
```

Your Feature (Required) 💡

To help us understand your product sense and creativity, **please design and implement at least one new, meaningful feature** that is not listed in this document.

Stretch Goals (Optional)

- Paginate the `GET /notes` API call.
- Implement a full-text search endpoint for notes (`GET /search?q=keyword`).
- Dockerize the application.
- Build a basic frontend to interact with the API.

Notes for Candidates

- **Candidates must submit the base URL of your deployed application** (e.g., `https://my-notes-app.render.com`).
  Our automated tests will use this URL to call your API endpoints by suffixing the path to the base URL like `https://my-notes-app.render.com/about`, `https://my-notes-app.render.com/login`.

- You'll be judged for edge case handling. So, ensure that the API endpoints have good validations.
- Use any language you're comfortable with.
- Use any database you're comfortable with, such as PostgreSQL, SQLite, etc.
- Keep the project simple, functional, and secure.
- You are free to use third-party libraries for things like JWT or password hashing.